

PASSGP · AI ORAL EXAMINER

Technical Documentation

Architecture, methodology & complete backend API reference

React + Vite

Node.js + Express

Supabase (Postgres)

OpenAI Realtime

Jotform

Vercel

Prepared for **PassGP**

Repository github.com/Waqas56jb/ai-oral-examiner-mvp

Document **Full technical reference (v1.0)**

Contents

1. Introduction & purpose
2. Technology stack
3. Repository & folder structure
4. System architecture & data flow
5. End-to-end methodology (the exam lifecycle)
6. The AI examiner & marking engine
7. Database schema
8. Backend API reference — every endpoint & payload
9. Integrations (OpenAI · Supabase · Jotform · Email)
10. Environment variables & configuration
11. Security model
12. Deployment & operations

1 Introduction & purpose

PassGP AI Oral Examiner is a web platform that lets medical candidates sit a realistic, spoken oral examination with an AI examiner. The candidate talks to the examiner in real time (voice), is marked against each exam's **own marking criteria** sourced from Jotform, and receives an elegantly formatted PDF report by email. Administrators (PassGP) curate the clinical case bank, define each exam's personality and marking standard, and review analytics — all from a dedicated admin console, with no code changes required for day-to-day operation.

The three applications

APP	AUDIENCE	TECH	LIVE URL
Chatbot (<code>client/</code>)	Candidates	React + Vite	<code>ai-oral-examiner-mvp-chatbot.vercel.app</code>
Admin console (<code>admin/</code>)	PassGP staff	React + Vite	<code>ai-oral-examiner-mvp-admin.vercel.app</code>
Backend API (<code>server/</code>)	Both apps	Node + Express	<code>ai-oral-examiner-mvp-backend.vercel.app</code>

Design principles

- **Secrets stay server-side.** The browser apps never hold API keys; the backend mints short-lived tokens.
- **Marking follows the source.** Scores come from each case's Jotform criteria (real totals), never a generic /10.
- **Configurable without code.** Examiner behaviour, mark schemes and pass standards are edited in the admin.
- **Single source of truth.** One Postgres database (Supabase) backs all three apps.

2 Technology stack

LAYER	TECHNOLOGY	WHY
Frontend (both apps)	React 18, Vite, React Router, Recharts, react-icons	Fast SPA build, charts for reports/analytics
Voice	OpenAI Realtime API over WebRTC	Low-latency two-way speech
Backend	Node.js, Express, OpenAI SDK	Thin secure API + AI orchestration

Database / Auth	Supabase (Postgres + Auth + RLS)	Managed DB, admin login, row-level security
Case source	Jotform Enterprise API	Existing clinical case bank
Email + PDF	nodemailer (SMTP) + pdf-lib / html2canvas + jsPDF	Report delivery
Hosting / CI	Vercel (3 projects, auto-deploy from GitHub <code>main</code>)	Zero-config deploys

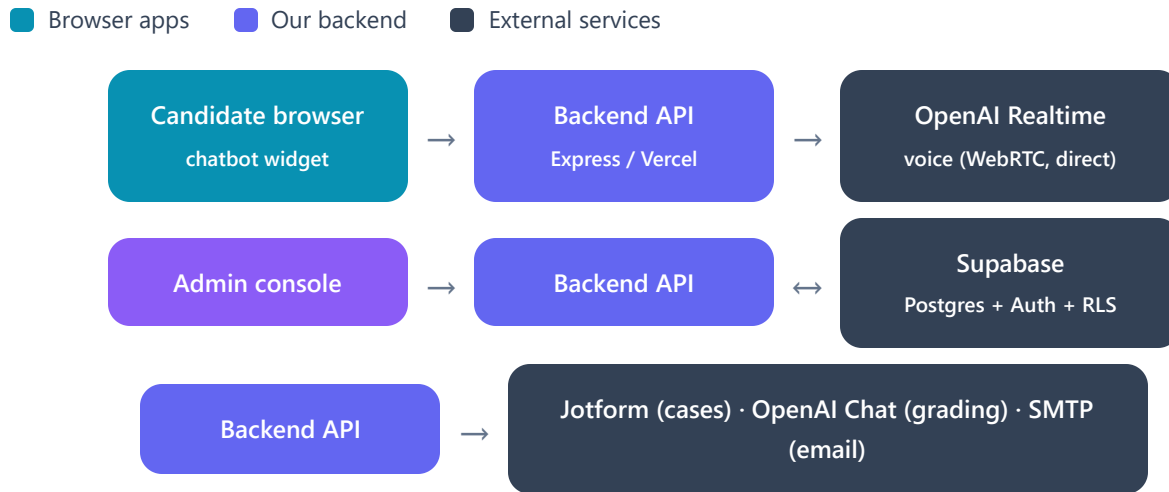
3 Repository & folder structure

A monorepo with three independently-deployable apps. Each app has its own `package.json` and `.env`.

```
ai-oral-examiner-mvp/
├─ server/                                Backend API (Node + Express)
│   ├─ server.js                          All HTTP endpoints (the API surface)
│   ├─ config.js                          Models, voice, CORS allow-list
│   ├─ email.js                           Send report emails (nodemailer SMTP)
│   ├─ report-pdf.js                      Fallback PDF report (pdf-lib)
│   ├─ prompts/
│   │   └─ examiner.js                    ★ The AI "brain": examiner + grader prompts
│   ├─ integrations/
│   │   └─ jotform.js                     Import & parse Jotform cases + marking
│   ├─ db/
│   │   ├─ supabase.js                   Server Supabase client (service key)
│   │   ├─ repo.js                       Database queries (cases, exams, sessions)
│   │   ├─ runSql.mjs                    CLI migration runner
│   │   └─ migrations/                   Schema history (001 ... 016)
│   └─ .env                               Secrets (git-ignored)
├─ client/                                Candidate chatbot (React + Vite)
│   ├─ src/pages/VoiceAgent.jsx           The live voice exam screen
│   ├─ src/pages/Report.jsx               Report + auto-email (charts → PDF)
│   ├─ src/services/realtimeClient.js     WebRTC connection to OpenAI
│   ├─ src/services/config.js             API base URL
│   └─ public/widget.js                   Kajabi embed script
├─ admin/                                 Admin console (React + Vite)
│   ├─ src/pages/Questions.jsx            Case bank (CRUD, CSV, import, preview)
│   ├─ src/pages/Training.jsx             Training panel (curate + bulk assign)
│   ├─ src/pages/ExamProfiles.jsx         Per-exam personality + mark scheme
│   ├─ src/pages/Sessions.jsx             Review + manual override
│   ├─ src/pages/Analytics.jsx            Charts + Power BI export
│   ├─ src/pages/AIConfig.jsx             Voice / model / role mode / focus
│   ├─ src/pages/Settings.jsx             Keys, email, integrations
│   └─ src/lib/api.js                     Authenticated fetch helpers
└─ docs/                                 This documentation
```

The file marked ★ — `server/prompts/examiner.js` — is where the examiner's behaviour and the marking logic live. Most "how the AI behaves" questions resolve here.

4 System architecture & data flow



Request lifecycle of a live exam

- **1. Token mint.** The chatbot calls `POST /api/realtime/session`. The backend builds the examiner prompt (case + personality + standard), asks OpenAI for a short-lived *ephemeral* key, and returns it. The real `OPENAI_API_KEY` never leaves the server.
- **2. Direct voice.** The browser opens a WebRTC connection *straight to OpenAI* using the ephemeral key — low latency, no audio through our server.
- **3. Transcript.** Speech-to-text events stream back over the data channel; the chatbot collects the transcript.
- **4. Grading.** On finish, the chatbot calls `POST /api/feedback`. The backend grades the transcript against the case's marking criteria (OpenAI Chat) and saves the session to Supabase.
- **5. Report + email.** The report page renders charts, captures the report to PDF, and calls `POST /api/email-report` to email it.

5 End-to-end methodology (the exam lifecycle)

#	STAGE	WHAT HAPPENS
1	Registration	Candidate enters name + email + chooses an exam (e.g. StAMPS, RACGP CCE). Exams shown come from <code>GET /api/exam-profiles</code> (only exams that have cases).
2	Case selection	The examiner greets, states how many cases are available, and asks "which case number, or 'any'?". A specific case can also be forced via the embed URL (<code>?caseId=</code> / <code>?formId=</code>).
3	Live oral exam	Two-way voice with the AI examiner. It plays examiner <i>and</i> patient (per the exam's role mode), probes to the exam's standard, and supports barge-in, a countdown timer and auto-termination.
4	Marking	The transcript is graded against the case's marking criteria and real total (e.g. 17), with the exam's mark scheme + standard deciding pass/fail. Killer/unsafe items force an automatic fail.
5	Report	A visual report (gauge, competency radar, domain bars, strengths, missed items) is shown and saved.
6	Email	The exact on-screen report (with charts) is emailed to the candidate as a PDF.
7	History	Candidates can look up their past attempts by email; admins see every session + analytics.

Mock circuits

Launching with `?mock=N` runs an N-station circuit back-to-back; each station is graded and saved separately, with a combined circuit summary at the end.

6 The AI examiner & marking engine

The examiner prompt is assembled at session time from: the chosen exam's **profile** (personality, role mode, mark scheme, standard), the selected **case** (scenario, patient script, questions, criteria), and global **AI config** (voice, difficulty, any admin override).

How marking reads the Jotform

- **SAQ / OSCE cases:** every point under "Correct possible answers" = **1 mark**; the total is the number of points (e.g. 17), not 10.
- **MCQ cases:** the `(Score: N)` annotations are summed.

- **Pass mark:** defaults to 50% of the case total; overridable per case, and the exam's **standard** guides borderline pass/fail.
- **Exam sessions:** because the candidate picks a case verbally, the grader is given the exam's cases (deterministic set), identifies which one the transcript covers, and marks against *that* case's scheme.

7 Database schema (Supabase Postgres)

exam_questions — the case bank

COLUMN	TYPE	MEANING
id	uuid	Primary key
external_ref	text	The Jotform form ID (matching key for CSV)
title	text	Case title
exam_type	text	Clinical category (or legacy exam tag)
pathway	text	The exam the case belongs to (set by admin)
stem	text	Clinical scenario
patient_script	text	Examiner-only patient role-play
marking_criteria	jsonb	Array of markable points
model_answer	text	Full correct answers
examiner_instructions / red_flags / killer_marks / feedback_points	text	Examiner-only marking material
total_marks / pass_mark / duration_seconds	int	Marking controls (real total)
status	text	draft · active · disabled · archived
is_active / in_training	bool	Live to candidates / in the curated training set

exam_profiles — per-exam configuration

`exam_key` , `label` , `examiner_instructions` (personality), `mode` (both/examiner/patient),
`mark_scheme` , `standard` , `enabled` .

exam_sessions — completed attempts

`candidate_name` , `candidate_email` , `pathway` , `exam_type` , `question_id` , `form_id` , `case_title` ,
`duration_sec` , `score` , `score_override` , `marks_awarded` , `total_marks` , `pass_fail` ,

`killer_failed` , `strengths` , `improvements` , `missed_items` , `unsafe_areas` , `reviewed` ,
`reviewer_note` .

session_turns

One row per spoken turn: `session_id` , `role` (examiner/candidate), `text` , `time_marker` .

app_settings

Key/value config: `ai_config` , `widget_theme` , `email_config` , `openai_fallback` , `analytics_api_key` .

admin_users

Admin accounts (`id` ↔ Supabase Auth, `email` , `role`).

8

Backend API reference

Base URL: `https://ai-oral-examiner-mvp-backend.vercel.app`. All bodies are JSON. Auth legend:

PUBLIC

no auth ·

ADMIN

Supabase JWT of an admin (Bearer token) ·

API KEY

shared key.

Public & candidate

GET `/api/health` **PUBLIC**

Service heartbeat.

```
200 → { "status":"ok", "service":"passgp-server", "chatModel":"gpt-4o-mini",
  "realtimeModel":"gpt-realtime" }
```

GET `/api/widget-theme` **PUBLIC**

The admin-selected voice-widget appearance.

```
200 → { "template": "cosmic" }
```

GET `/api/exam-profiles` **PUBLIC**

Exams a candidate can choose (only those with cases). Drives the registration dropdown.

```
200 → { "exams":[ { "exam_key":"StAMPS (ACRRM)", "label":"StAMPS (ACRRM)", "caseCount":2 }
  ] }
```

GET `/api/candidate/history?email={email}` **PUBLIC**

A candidate's own past sessions by email (no transcript).

```
200 → { "sessions":[ { "id","created_at","exam_type","pathway","case_title",
  "score","score_override","pass_fail","marks_awarded","total_marks",
  "summary","strengths","improvements","missed_items","unsafe_areas" } ] }
```

GET /api/mock/circuit?count={n}&pathway={p}&exam={e} **PUBLIC**

Generate an N-station mock circuit from the training set.

```
200 → { "stations":[ { "station":1,
"questionId","title","examType","pathway","durationSeconds":480 } ] }
409 → { "maintenance":true, "error":"...", "stations":[] } // no trained cases
```

POST /api/realtime/session **PUBLIC**

Mint a short-lived OpenAI Realtime token + build the examiner prompt.

Request

```
{ "candidateName":"Dr Khan", "examType":"StAMPS (ACRRM)", "questionId":null, "formId":null
}
```

Response

```
200 → { "value":"<ephemeral key>", "model":"gpt-realtime",
"webrtcUrl":"https://api.openai.com/v1/realtime/calls",
"questionId","formId","questionTitle","examType","durationSeconds":900 }
409 → { "maintenance":true, "error":"..." } // nothing trained for this exam
```

POST /api/chat **PUBLIC**

Text (typed) examiner — Server-Sent Events stream.

```
body → { "messages":[{ "role":"user","content":"..." }], "candidateName","examType" }
stream → event: delta { "text":"..." } ... event: done { "ok":true }
```

POST /api/feedback **PUBLIC**

Grade a finished transcript and (optionally) save the session.

Request

```
{ "transcript": [{ "role": "examiner|candidate", "text": "...", "time": "02:14" }],
  "examType": "StAMPS (ACRRM)", "questionId": null, "formId": null,
  "durationSec": 423, "save": true,
  "candidateName": "Dr Khan", "candidateEmail": "dr@x.com", "pathway": "StAMPS (ACRRM)" }
```

Response (scored report)

```
200 → { "candidate_name", "overall_score", "clinical_reasoning", "diagnosis",
  "management", "communication", "marks_awarded": 15, "total_marks": 17,
  "pass_mark": 9, "pass_fail": "Pass", "killer_failed": false,
  "strengths": [], "weaknesses": [], "missed_items": [], "unsafe_areas": [],
  "recommendations": [], "examiner_comments": "...",
  "score": 88, "result": "Competent", "domains": [...], "sessionId": "uuid" }
```

POST /api/email-report **PUBLIC**

Email the candidate the rendered report PDF (with charts).

```
body → { "candidateEmail", "candidateName", "examType",
  "pdfBase64": "data:application/pdf;base64,...", "report": {...} }
200 → { "sent": true } | { "sent": false, "reason": "not configured" }
```

All require header `Authorization: Bearer <admin JWT>`. The backend verifies the token *and* that the user exists in `admin_users`.

GET `/api/admin/questions`

All cases (light fields, paginated server-side).

```
200 → { "questions": [{  
  "id", "title", "exam_type", "pathway", "in_training", "is_active", "status" }], "total": 2941 }
```

GET `/api/admin/questions/:id`

```
200 → { "question": { ...all case fields... } }
```

POST `/api/admin/questions`**PUT** `/api/admin/questions/:id`**PATCH**`/api/admin/questions/:id`

Create / replace / partially update a case. Body is the whitelisted case fields:

```
{ "title", "exam_type", "pathway", "candidate_instructions", "stem", "patient_script",  
  "marking_criteria": [...], "model_answer", "examiner_instructions", "red_flags",  
  "feedback_points", "total_marks", "pass_mark", "duration_seconds", "killer_marks",  
  "status", "is_active", "in_training" }  
200 → { "question": {...} }
```

DELETE `/api/admin/questions/:id`

```
200 → { "ok": true }
```

POST /api/admin/questions/bulk-update

Update a field on many cases (e.g. assign an exam).

```
body → { "ids":["uuid",...], "patch":{"pathway":"RACGP CCE" } } → { "updated":240 }
```

POST /api/admin/questions/bulk-delete

```
body → { "ids":["uuid",...] } → { "deleted":12 }
```

POST /api/admin/exams/assign-csv

Bulk-assign cases to exams from a CSV (matched by Jotform form ID, then title).

```
body → { "rows":[ { "ref":"250036...", "title":"Case G240", "exam":"AMC" } ] }  
200 → { "matched":612, "updated":612, "byExam":{"AMC Clinical":240, ... }, "unmatched":[...]  
}
```

POST /api/admin/questions/bulk

Bulk upsert cases from a full-field CSV import.

```
body → { "rows":[ { ...case fields... } ] } → { "inserted","updated","failed","total" }
```

GET /api/admin/questions/export

All cases with every field (for CSV export).

```
200 → { "questions":[ { ...full fields... } ], "total":2948 }
```

POST /api/admin/training/set

Add / remove cases from the curated training set.

```
body → { "ids":["uuid",...], "in_training":true } → { "updated":8 }
```

GET /api/admin/jotform/forms

200 → { "forms":[{"id","title","count","status" }], "total":2942 }

POST /api/admin/jotform/import

Import a batch of Jotform forms (parses marking criteria + real total).

body → { "forms":[{"id","title" }] } → { "imported","updated","failed","total" }

GET /api/admin/exam-profiles **PUT** /api/admin/exam-profiles/:examKey

List exams with case counts / upsert a profile.

PUT body → { "label","examiner_instructions","mark_scheme","standard","mode","enabled" }
200 → { "profile": {...} }

GET /api/admin/analytics/export

Flat, one-row-per-attempt analytics (JSON) for the in-app export.

200 → { "rows":[{"session_id","datetime","candidate_name","candidate_email","pathway",
"category","case_id","case_title","duration_minutes","marks_awarded","total_marks",
"score_percent","final_pass_fail","killer_failed","missed_rubric_items" }],
"total":N }

POST /api/admin/auth/admin-reset-password

A superadmin resets another admin's password.

body → { "email","password" } → { "ok":true }

Analytics API (external)

GET**/api/analytics/export?key={KEY}&format=csv|json****API KEY**

For Power BI / BI tools to pull directly. Key via `?key=` or header `x-api-key`.

200 → CSV (default) or { "rows":[...], "total":N } when format=json
401 → { "error":"Invalid API key" } · 503 → not configured

9 Integrations

SERVICE	ENDPOINT / SDK	NOTES
OpenAI Realtime	<code>/v1/realtime/client_secrets</code> + WebRTC <code>/v1/realtime/calls</code>	Ephemeral token minted server-side; browser streams voice directly.
OpenAI Chat	OpenAI SDK <code>chat.completions</code>	JSON-mode grading; transparent retry on an admin fallback key.
Supabase	<code>@supabase/supabase-js</code> + pooler for migrations	Service key server-side; anon key in browser; RLS enforced.
Jotform	<code>/user/forms</code> , <code>/form/{id}/questions</code>	Paginated import; parses scenario, questions, criteria, marks.
Email	nodemailer SMTP	Sender + app password from env; default host <code>smtp.gmail.com</code> (override with <code>SMTP_HOST</code>).

10 Environment variables & configuration

Backend (server / Vercel backend project)

VARIABLE	PURPOSE
<code>OPENAI_API_KEY</code>	OpenAI (voice + grading)
<code>SUPABASE_URL</code> , <code>SUPABASE_SECRET_KEY</code> , <code>SUPABASE_PUBLISHABLE_KEY</code>	Database + Auth
<code>JOTFORM_API_KEY</code>	Case import
<code>Owner_Email</code> , <code>App_Password</code>	Report email sender
<code>ALLOWED_ORIGINS</code>	Extra CORS origins (PassGP / Kajabi domains)
<code>ANALYTICS_API_KEY</code>	Protects the Power BI export
<code>SMTP_HOST</code> (optional)	Non-Gmail SMTP host

Frontends (admin & chatbot)

`VITE_SUPABASE_URL` · `VITE_SUPABASE_ANON_KEY` · `VITE_API_BASE` (the backend URL).

All `.env` files are git-ignored. Production values live in the Vercel project settings, not in the repository.

11 Security model

- **Secret isolation.** API keys exist only on the backend. The browser receives short-lived ephemeral tokens for voice.
- **CORS allow-list.** Only the PassGP apps (and configured domains) may call the API.
- **Row-Level Security.** Direct writes to the case bank from the browser are revoked; all create/edit/delete go through authenticated backend endpoints (service role).
- **Admin auth.** Supabase Auth; every admin endpoint checks the JWT *and* `admin_users` membership; superadmin role for sensitive actions.
- **Password reset.** Secure email-link flow (no password set from email alone).
- **Analytics API.** Separate shared key, never the database key.

12 Deployment & operations

- **CI/CD.** GitHub `main` → Vercel auto-deploys all three projects.
- **Migrations.** SQL files in `server/db/migrations/`, applied with `node db/runSql.mjs <file>` via the Supabase pooler.
- **Re-import.** Admin → Questions → "Import from Jotform" → Import all re-reads every case's marking scheme.
- **Admin login.** `admin@passgp.au` (reset via the login page → emailed link).

Change behaviour without code: exam personalities / mark schemes / pass standards → *Exam Profiles*; voice, models, role mode → *AI Configuration*; which cases are live → *Training Panel*; keys & email → *Settings*.